Hochschule Zittau/Görlitz
UNIVERSITY OF APPLIED SCIENCES

Polling & Long Polling

Web Engineering 1 (319300)

Vorlesung 04

M.Sc. Christopher-Manuel Hilgner

Hochschule Zittau/Görlitz

christopher.hilgner@stud.hszg.de

2026-10-27

Agenda

- Polling & Long Polling
- Web Sockets

— Agenda

- Begrüßung, kurzer Rückblick auf Vorlesung 03 (Fetch-API, async/await)
- Lernziel: Die drei Grundansätze kennen und deren Vor- und Nachteile begründen können
- Zeitplan: ca. 30 Minuten Polling/Long Polling, Rest Web Sockets

Polling & Long Polling

Das Problem

- HTTP ist ein **Request-Response-Protokoll**: Der Client fragt, der Server antwortet
- Der Server kann von sich aus **nichts** an den Browser schicken
- Frage: Wie erfährt die Seite von neuen Daten (Chatnachricht, Benachrichtigung, Auftragsstatus)?

Drei Ansätze mit steigender Komplexität:

Ansatz	Idee
Polling	Client fragt in festen Abständen nach
Long Polling	Server hält die Anfrage offen, bis es etwas Neues gibt
Server-Sent Events, Web Sockets	Dauerhafte Verbindung (kommt später)

Einstiegsfrage an das Auditorium: Wer hat schon mal einen Chat im Browser benutzt – wie kommt die neue Nachricht eigentlich auf den Bildschirm? Die Tabelle bewusst nur kurz vorstellen, Details folgen auf den nächsten Folien.

Polling

- Der Client sendet in regelmäßigen Abständen eine Anfrage, egal ob es etwas Neues gibt
- Einfachste Variante
- Funktioniert mit jedem Server und jedem Proxy

Analogie: Der Client klingelt alle paar Sekunden an der Tür und fragt „Gibt's was Neues?“. Schon hier andeuten: Die meisten Anfragen kommen leer zurück – genau das ist der Nachteil, der auf der Bewertungsfolie auftaucht. Robustheit ist der eigentliche Verkaufspunkt: keine Spezialtechnik nötig, läuft auch in restriktiven Firmennetzwerken.

Polling

Beispiel

```JavaScript
async function pollStatus() {
  try {
    const response = await fetch("/api/orders/42/status");
    if (!response.ok) throw new Error("HTTP ${response.status}");
    const { status } = await response.json();
    updateUI(status);
  } catch (error) {
    console.error(error);
  }
}
setInterval(pollStatus, 5000); // alle 5 Sekunden
```


Code gemeinsam durchgehen: fetch, Fehlerprüfung über response.ok, Destrukturierung aus der JSON-Antwort, updateUI als Blackbox lassen. Frage ans Auditorium: Was passiert, wenn der Server länger als 5 Sekunden braucht? – leitet direkt zur nächsten Folie über.

Polling Umsetzung

Problem von `setInterval`: Dauert eine Antwort länger als das Intervall, überlappen sich beide Anfragen.

Besser: rekursives `setTimeout`. Die nächste Anfrage startet erst, wenn die letzte fertig ist.

Weitere Verbesserungen:

- Polling stoppen, wenn der Tab im Hintergrund ist (`document.visibilityState`, Event `visibilitychange`)
- Bei Fehlern das Intervall vergrößern (Exponential Backoff: 5s, 10s, 20s, …)
- Abbruch über `AbortController`, z.B. beim Verlassen der Seite
- Serverseitig `ETag`/`If-None-Match` nutzen: Antwort `304 Not Modified` spart Daten

Kernproblem kurz an der Tafel skizzieren: zwei zeitlich überlappende Anfragen bei setInterval. Die Verbesserungen der Liste nur streichen – jede davon wird im Praktikum relevant. ETag/304 an die HTTP-Grundlagen anbinden, die Studenten kennen Conditional Requests bereits.

Polling Umsetzung

```javascript
async function poll() {
  try {
    const response = await fetch("/api/orders/42/status");
    if (!response.ok) throw new Error("HTTP ${response.status}");
    const { status } = await response.json();
    updateUI(status);
  } catch (error) {
    console.error(error);
  } finally {
    setTimeout(poll, 5000);
  }
}
poll();
```


Gegenüberstellung mit dem ersten Beispiel: Der Unterschied steckt komplett im finally-Block – setTimeout startet erst, wenn die vorherige Anfrage abgeschlossen ist (auch im Fehlerfall). Serielle Ausführung statt möglicher Parallelität.

Bewertung

Vorteile:
- Sehr einfach zu implementieren
- Funktioniert überall (Proxys, Firewalls, Caches)
- Server bleibt zustandslos

Nachteile
- Latenz bis zu einem Intervall (bei 5s im Schnitt 2.5s Verzögerung)
- Viele unnötige Requests, wenn sich selten etwas ändert
- Kurzes Intervall = Extra Last auf Server und Netz
- Langes Intervall = Träge UI

Typischer Einsatz: Dashboards, Statusabfragen von langlaufenden Jobs, wenn Verzögerungen von Sekunden egal sind

Rechenfrage ans Auditorium: Warum im Schnitt 2,5 Sekunden Verzögerung bei einem 5-Sekunden-Intervall? (Event tritt gleichverteilt zwischen zwei Polls auf.) Trade-off kurzes/langes Intervall an den Nachteilen festmachen. Bekanntes Beispiel als Einsatzfall: Statusseite eines Deployments oder Paketverfolgung.

Long Polling

Long Polling

Idee: Der Server beantwortet die Anfrage nicht sofort, sondern hält sie offen, bis es etwas Neues gibt (oder ein Timeout erreicht ist). Danach öffnet der Client sofort die nächste Anfrage.

Ablauf

1. Client sendet Anfrage
2. Server wartet (z.B. bis zu 30s)
3. Neue Daten da: Server antwortet sofort. Timeout: Server antwortet leer (z.B. 204 No Content)
4. Client verarbeitet die Antwort und startet direkt die nächste Anfrage

Mentalmodell: Der Server „schläft“ im Request-Handler ein und wird von den neuen Daten geweckt. Die vier Schritte nummeriert durchgehen. Wichtig: Das Timeout (leere 204er-Antwort) sorgt dafür, dass Proxys die Verbindung nicht vorher abwürgen – dazu mehr auf den nächsten Folien.

Long Polling

```JavaScript
async function longPoll(lastId = 0) {
  while (true) {
    try {
      const response = await fetch("/path?after=${lastId}");
      if (response.status == 200) {
        const messages = await response.json();
        messages.forEach(show);
        lastId = messages.at(-1).id;
      }
      // 204 = Timeout ohne Neuigkeiten, erneut fragen
    } catch (error) {
      console.error(error);
      await new Promise((resolve) => setTimeout(resolve,
      5000)); // Pause bei Fehler
```

Long Polling

```
14        }
15      }
16    }
17    longPoll();
```

Wichtig: Der Client merkt sich, was er schon hat (`lastId`), damit keine Nachricht zwischen zwei Anfragen verloren geht.

Long Polling
— Long Polling

Fokus auf lastId: Das Cursor-Prinzip schließt die Lücke, die zwischen zwei Anfragen entsteht – eine Nachricht, die genau dazwischen am Server ankommt, wird mit dem nächsten Poll über „after“ noch geliefert. Kurz auf den Fehlerzweig eingehen: 5-Sekunden-Pause ist hier bereits einfachstes Backoff.

Long Polling bewertet

Vorteile:

- Fast sofortige Zustellung
- Weniger leere Requests als Polling
- Läuft über normales HTTP

Nachteile:

- Der Server hält viele offene Verbindungen
- Proxys und Load Balancer beenden lange Verbindungen (Timeout oft 30 bis 60s)
- Pro Domain gibt es Verbindungslimits (HTTP/1.1: meist 6 pro Browser)
- Zwischen zwei Anfragen entsteht kurz eine „Lücke“

Kontrast zu Polling herausarbeiten: Latenz fast null, aber der Preis sind dauerhaft offene Verbindungen auf dem Server. Das Verbindungslimit von 6 pro Domain ist praktisch relevant: mehrere Tabs und Services teilen sich dasselbe Budget. Die „Lücke“ wurde auf der Code-Folie schon mit dem lastId-Cursor gelöst.

Long Polling bewertet

- Server-Timeout kürzer als der Proxy-Timeout wählen
- Server muss viele wartende Anfragen effizient halten können (asynchrones/nicht blockierendes I/O, bei Thread-pro-Request-Modellen wird das sehr schnell eng)
- Bei Fehlern immer mit Pause und Backoff neu verbinden, sonst entsteht eine „Thundering Herd“, wenn der Server wieder erreichbar ist

Faustregel zuerst: Server-Timeout unterhalb des Proxy-Timeouts wählen, sonst antwortet der Proxy mit einem Fehler, bevor der Server überhaupt dran war. „Thundering Herd“ erklären: Nach einem Ausfall feuern alle Clients gleichzeitig sofort wieder los – nur die Pause mit Backoff verteilt die Last.

Vergleich und Ausblick

	Polling	Long Poll.	SSE	WebSocket
Richtung	Client → Server	Client → Server	Server → Client	beidseitig
Latenz	Intervall	gering	gering	gering
Verbindung	neu pro Anfrage	neu pro Antwort	dauerhaft	dauerhaft
Protokoll	HTTP	HTTP	HTTP (`EventSource`)	eigenes (`ws://`, `wss://`)
Aufwand	sehr gering	mittel	gering	höher

Tabelle zeilenweise durchgehen. Die Spalten SSE und WebSocket sind bewusst noch Blackbox – Vorschau auf die kommende Vorlesung. An der Zeile „Richtung“ festigen: Bei Polling und Long Polling initiiert immer der Client die Kommunikation, bei SSE dreht sich das um.

Vergleich und Ausblick

Faustregel:

- Selten ändernde Daten, Sekunden egal: **Polling**
- Neuigkeiten sollen schnell ankommen, nur Server → Client: **SSE** (oder Long Polling als Fallback)
- Echte Zwei-Wege-Kommunikation (Chat, Spiele, Kollaboration): **WebSocket**

Kurzer Ausblick auf SSE:

```javascript
const source = new EventSource("/api/events");
source.onmessage = (event) =>
console.log(JSON.parse(event.data));
```


Kurzquiz zu den Faustregeln: Welche Technik für (a) Paketverfolgung, (b) Live-Ticker, (c) Multiplayer-Spiel? Das EventSource-Snippet nur als Ausblick zeigen, kein Code-Walkthrough – Details folgen in der nächsten Vorlesung.

Typische Fehler

- `setInterval` mit `async`-Funktion: Anfragen überlappen sich, wenn der Server langsam ist
- **Kein Abbruch**: Polling läuft weiter, obwohl die Seite oder Komponente längst weg ist (Memory Leak, vgl. Folie zu Garbage Collection)
- **Zu kurze Intervalle**: 1-Sekunden-Polling von 1000 Nutzern sind 1000 Requests pro Sekunde
- **Endlosschleife ohne Pause bei Fehlern**: Der Client hämmert dann den Server mit Fehlversuchen zu

Typische Review-Funde – gut als Abschlussdiskussion geeignet: Welchen der vier Fehler habt ihr schon selbst gebaut? Beim Stichwort Memory Leak an die Vorlesung zu Garbage Collection anknüpfen. Bei „1-Sekunden-Polling von 1000 Nutzern“ die Hochrechnung kurz vorführen (1000 Requests/s).

Web Sockets

Welches Problem wird gelöst?

Technische Übersicht

Welches Problem wird gelöst?

- Klassisches HTTP ist ein Request-Response-Protokoll
- Nur der Client intialisiert Kommunikation
- Für real-time Anwendungen (Chats, Live Dashboards, Multiplayer Spiele, ...) würde sonst Polling benötigt werden
- Wiederholte HTTP Requests wären verschwenderisch und *laggy*

Technische Übersicht

Welches Problem wird gelöst?

- Klassisches HTTP ist ein Request-Response-Protokoll
- Nur der Client intialisiert Kommunikation
- Für real-time Anwendungen (Chats, Live Dashboards, Multiplayer Spiele, ...) würde sonst Polling benötigt werden
- Wiederholte HTTP Requests wären verschwenderisch und *laggy*

Was ist eine WebSocket?

Technische Übersicht

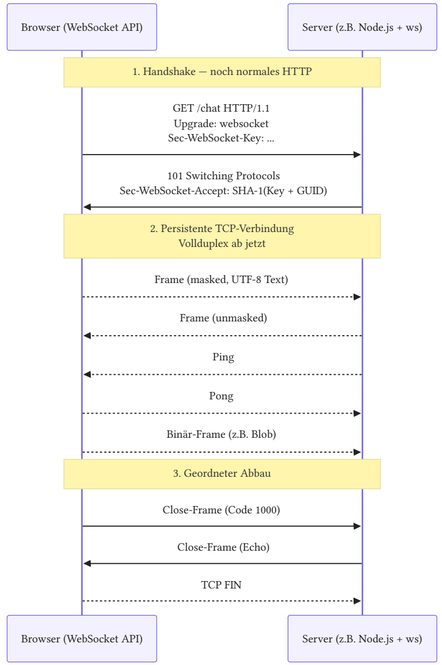
Welches Problem wird gelöst?

- Klassisches HTTP ist ein Request-Response-Protokoll
- Nur der Client intialisiert Kommunikation
- Für real-time Anwendungen (Chats, Live Dashboards, Multiplayer Spiele, ...) würde sonst Polling benötigt werden
- Wiederholte HTTP Requests wären verschwenderisch und *laggy*

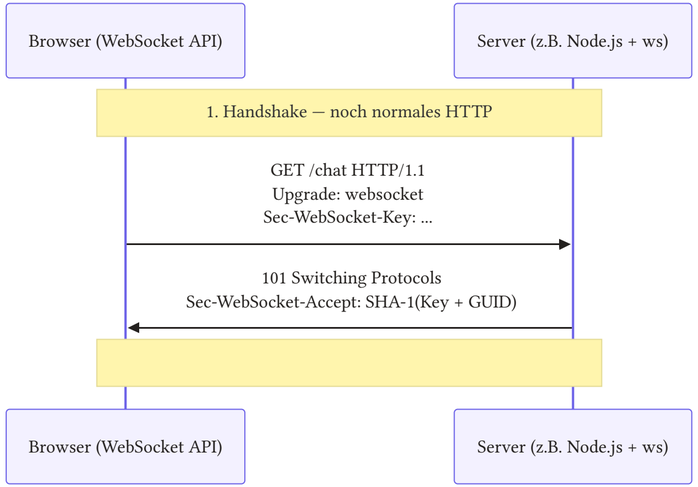
Was ist eine WebSocket?

- Eine einzelne persistente TCP Verbindung
- Wird mit HTTP konfiguriert bzw. verhandelt
- Wird danach zu einer full-duplex bzw. einem bidirektionalem Kanal (definiert nach RFC 6455)
- Beide Seiten können Nachrichten schicken mit minimalem Overhead

Ablauf


Browser (WebSocket API)
Server (z.B. Node.js + ws)
1. Handshake — noch normales HTTP
GET /chat HTTP/1.1
Upgrade: websocket
Sec-WebSocket-Key: ...
101 Switching Protocols
Sec-WebSocket-Accept: SHA-1(Key + GUID)
2. Persistente TCP-Verbindung
Vollduplex ab jetzt
Frame (masked, UTF-8 Text)
Frame (unmasked)
Ping
Pong
Binär-Frame (z.B. Blob)
3. Geordneter Abbau
Close-Frame (Code 1000)
Close-Frame (Echo)
TCP FIN
Browser (WebSocket API)
Server (z.B. Node.js + ws)

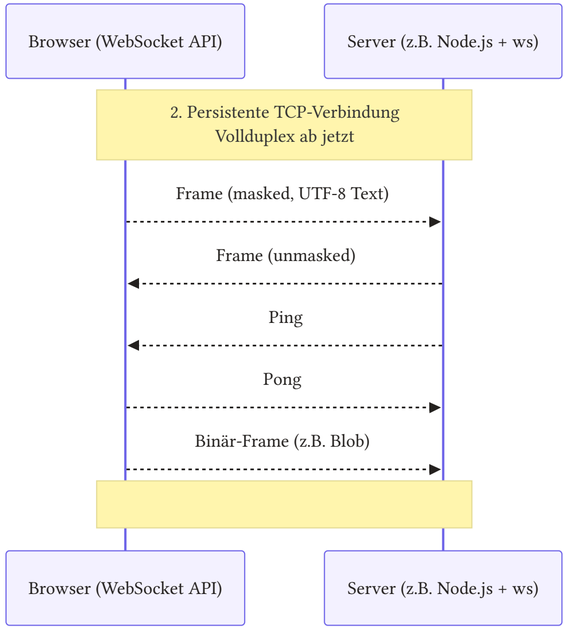
Ablauf - Phase 1



Phase 1

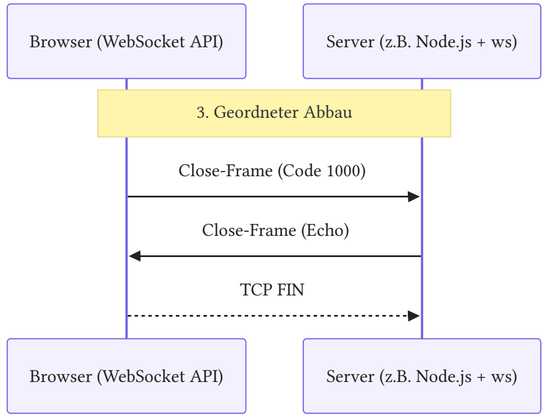
- Die Verbindung beginnt als **ganz normale HTTP-Anfrage**: ein `GET` mit Sonderheadern:
  - `Upgrade: websocket` und `Connection: Upgrade`: Forderung nach Protokollwechsel
  - `Sec-WebSocket-Key: <Base64>`: zufälliger 16-Byte-Wert des Clients
  - `Sec-WebSocket-Versiom: 13`: aktuelle Protokollversion (RFC 6455)
- Der Server antwortet mit `101 Switching Protocols` und `Sec-WebSocket-Accept`: der SHA-1-Hash aus `Key + festem GUID`, Base64-kodiert
- **Zweck des Hashes:** Der Client prüft den Accept-Wert. Da ein HTTP-Server ohne WebSocket-Verständnis den Header nie korrekt berechnen kann, ist garantiert, dass Gegenüber wirklich ein WebSocket-Endpunkt ist (kein versehentlicher Proxy-Kontakt)
- **Ab hier ist HTTP beendet**- Dieselbe TCP-Verbindung wird weiterverwendet, aber sie spricht im WebSocket-Frameprotokoll

Ablauf - Phase 2



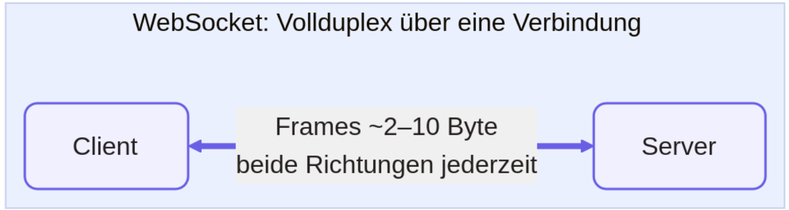
- **Ein TCP-Socket, beide Richtungen, jederzeit:** Weder Client noch Server müssen auf eine Anfrage warten. Der Server kann von sich aus Nachrichten pushen, der zentrale Unterschied zum Request-Response-Modell.
- **Frames statt Header:** Jede Nachricht besteht aus Frames mit nur **2-10 Byte Overhead** (gegenüber 500 Byte HTTP-Header pro Polling-Request). Geringe Latenz, geringer Overhead.
- **Maskierung (nur Client → Server):** Jeder Client-Frame wird mit einem zufälligen **4-Byte-XOR-Schlüssel** maskiert. Grund: Schutz vor Middleware-Proxys, die maskierte Daten nicht als HTTP-Header missinterpretieren können (Cache-Poisoning-Abwehr aus der Geschichte des Protokolls). Server-Frames sind unmaskiert.
- **Frame-Typen**
  - Text — UTF-8, typischer Fall für JSON-Nachrichten
  - Binary — Blobs, ArrayBuffer, z. B. Bilder oder Audio
  - Ping/Pong — Herzschlag: der Server (oder Client) sendet `Ping`, der Partner antwortet sofort mit `Pong`. Hält NAT-Timeouts und Proxys davon ab, die stille Verbindung abzubauen. Erwartet spätestens nach 30 s Abbruch.
- **Fragmentation:** Große Nachrichten dürfen in mehrere Kontinuations-Frames zerlegt werden (relevant für Streaming großer Binärdaten).

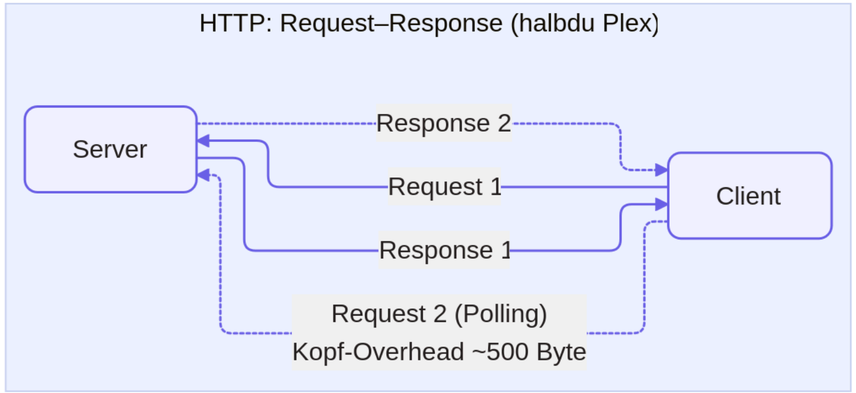
Ablauf - Phase 3



- Ein Beteiligter sendet einen **Close-Frame**, meist mit Statuscode:
  - `1000`: normaler Abschluss („Connection closed normally“)
  - `1001`: Endpunkt geht offline
  - `1006`: abrupter Abbruch ohne Close-Frame (Netzwerkfehler), wichtig für die Reconnect-Logik
- Der Partner **echo’t** den Close-Frame, erst danach schließt der Initiator die TCP-Verbindung.
- **Warum der Aufwand?** Beide Seiten wissen sicher, dass das Ende gewollt war, und können noch ausstehende Frames flushen. Ein fehlender Close-Frame signalisiert Fehlerfall → Reconnect.

HTTP vs. Web Sockets





Kernaussage: Beide Modelle laufen über TCP und können dieselben Daten transportieren. Der Unterschied liegt in **Wer initiieren darf** und **was jede Nachricht kostet**. HTTP erzwingt ein Frager-Antwort-Spiel mit hohem Wiederholungsaufwand. WebSocket zahlt einmalig für den Handshake und bekommt danach eine schlanke, in beiden Richungen offene, Leitung.

Der Handshake

Die Verbindung startet mit einer normalen HTTP GET Request mit einem `Upgrade` Header:

```
1   GET /chat HTTP/1.1
2   Host: exmaple.com
3   Upgrade: websocket
4   Connection: Upgrade
5   Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
6   Sec-WebSocket-Version: 13
```

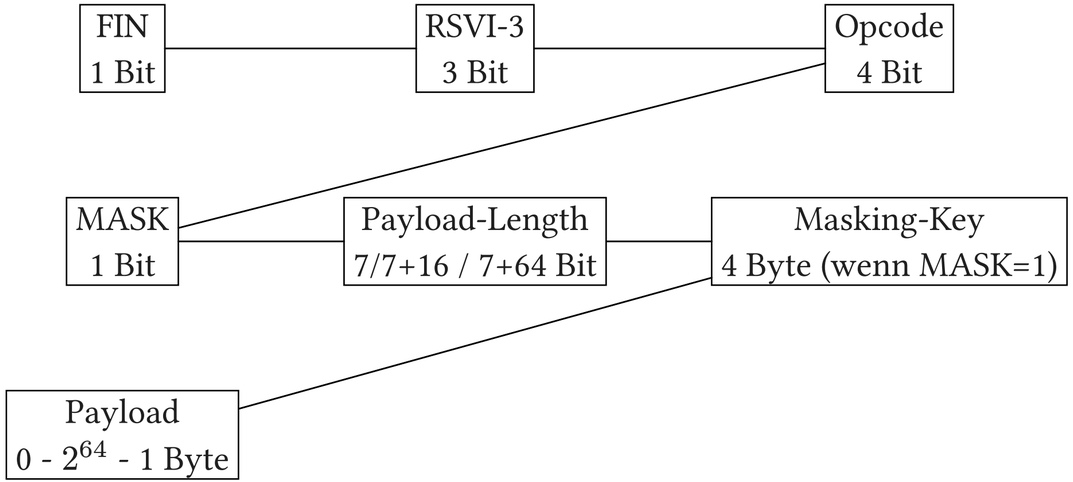
- Der Server antwortet mit `101 Switching Protocols` und einem `Sec-WebSocket-Accept` Value
- Beweis dafür, dass der Server WebSockets benutzen kann
- HTTP ist jetzt fertig → TCP übernimmt mit dem **WebSocket framing protocol**

Protokoll Komponenten

- **Frames:**
  - Client → Server Frames müssen maskiert werden (4-byte XOR Key als Sicherheitsmaßnahme)
  - Server → Client Frames werden nicht maskiert
- **Frame types:** Text (UTF-8), Binary, Ping/Pong (keep-alive), Close
- **Fragmentation:** Große Nachrichten können in mehrere Frames aufgeteilt werden
- **Closing:** Handshake um die Verbindung ordentlich zu schließen (close frame + TCP FIN), Verbindung wird sonst einfach abgebrochen

Aufbau eines Frames

- Jede Nachricht nach dem Upgrade ist ein **Frame**
- Frame-Format ist binär definiert: Header ist zwischen 2 und 14 Byte groß



Komponenten

FIN (1 Bit): Endfragment-Flag:

- `1` = die Nachricht ist vollständig in diesem Frame
- `0` = es folgen Kontinuations-Frames (Opcode `0`) bis ein Frame mit `FIN=1` kommt

RSV1-RSV3 (je 1 Bit): reserviert, müssen `0` sein, **außer** eine Erweiterung ist ausgehandelt

Komponenten

Opcode (4 Bit): Typ des Frames

Opcode	Frame	Bedeutung
`0x0`	Continuation	Fortsetzung einer fragmentierten Nachricht
`0x1`	Text	UTF-8-Nachricht, als Grenzes dekodierbar
`0x2`	Binary	beliebige Binärdaen (ArrayBuffer/Blob)
`0x8`	Close	leitet den geordneten Abbau ein
`0x9`	Ping	Herzschlag-Anfrage
`0xA`	Pong	Herzschlag-Antwort
`0x3-0x7, 0xB-0xF`	reserviert	ungültig im normalen Betrieb

Komponenten

MASK (1 Bit): muss bei allen Client-Frames `1` sein und `0` bei Server-Frames. Ein Server, der einen unmaskierten Client-Frame erhält (oder umgekehrt maskierte vom Server), trennt die Verbindung (Protokollfehler 1002).

Payload-Länge (7 Bit + optional): drei Stufen:

1. Länge `0-125`: passt direkt in die 7 Bit, kein Zusatzfeld
2. Wert `126`: die nächsten **2 Byte** (16 Bit, bis 65.535) tragen die echte Länge
3. Wert `127`: die nächsten **8 Byte** (64 Bit) tragen die echte Länge (bis 16 Exbibyte). Der Wert muss in minimaler Form kodiert sein: Länge 100 mit 8-Byte-Kodierung ist ein Protokollverstoß.

Komponenten

Masking-Key (4 Byte, bei MASK=1): ein zufälliger XOR-Schlüssel. Das Payload wird byteweise mit `payload[i] XOR key[i mod 4]` maskiert. Der Empfänger (Server) demaskiert identisch. Beachte: **Der Frame-Header wird nie selbst maskiert.**

Payload: die eigentlichen Daten, 0 Byte bis zur 64-Bit-Grenze.

Konkretes Beispiel (Server → Client)

```
1    81 05 48 61 6C 6C 6F
```

Byte(s)	Hex	Binär	Bedeutung
1	`81`	`1000 0001`	FIN=1 (eine komplette Nachricht), RSV=000, Opcode 0x1 = Text-Frame
2	`05`	`0000 0101`	MASK=0 (Server sendet nie maskiert), Payload-Länge = 5
3-7	`48 61 6C 6C 6F`		Payload: „Hallo“ in UTF-8

Konkretes Beispiel (Server → Client)

Im Browser:

- WebSocket-API des Browsers liefert Header
- Erkennt Opcode `0x1` (Text)
- Dekodiert den Payload als UTF-8
- Übergibt Ergebnishandler

```JavaScript
socket.onmessage = (event) => {
  event.data; // "Hallo"
  event.origin; // "wss://example.com"
  event.lastEventId; // "" (nur bei SSE relevant)
  event.ports; // []
}
```

Beispiel für mehrere Frames (Fragmentierung)

Frame 1 `FIN=0, Opcode=0x1, Payload="Web "` `01 04 57 65 62 20`
Frame 2 `FIN=1, Opcode=0x0, Payload="Engineering"` `80 0B 45 6E …`

- Erst mit dem zweiten Frame ist die Nachricht vollständig
- Wichtig: Der Opcode (`0x1` Text) steht nur im ersten Frame
- Alle Continuation-Frames tragen nur `0x0`
- Ein anderer Opcode im Continuation-Frame wäre Protokollverstoß

Server & Client API

```JavaScript
const socket = new WebSocket(
  "wss://example.com/chat"
);
socket.onopen = () => socket.send("Hello");
socket.onmessage = (event) => console.log(event.data);
socket.onclose = (event) => console.log(event.code, event.reason);
socket.onerror = (e) => console.error(e);
```

```JavaScript
import { WebSocketServer } from "ws";
const wss = new WebSocketServer({ port: 8080 });

wss.on("connection", (ws) => {
  ws.on("message", (data) => wss.clients.forEach(c =>
  c.send(data)));
});
```

Verbindungen zu vorherigen Themen

Event Loop: `onmessage` Callbacks sind normale Tasks, Nachrichten die ankommen werden in die Queue gelegt und asynchron abgearbeitet (wie `setTimeout`/`Promise`)

Promises/async-await: Connection, Reconnection und send ackowledgements werden in Promises verpackt

Vergleich zu Alternativen

	Polling	Long Polling	SSE	WebSockets
Richtung	Client → Server	Client → Server	Server → Client	**Bidirektional**
Latenz	Hoch	Mittel	Niedrig	**geringste**
Overhead	komplette HTTP Header	komplette HTTP Header	wenig	**2-10**
Protokoll	HTTP	HTTP	HTTP	eigenes Protokoll

Fun-Fact: Server-Side Events (SSE) wurden wieder populär für AI Streaming (z.B. ChatGPT Token Streams)